# UNIVERSIDAD DE GUADALAJARA

Licenciatura en Inteligencia Artificial
y Ciencia de Datos

# Proyecto Final

## Arquitectura de Sistemas de Inteligencia Artificial

Diseño e implementación de
sistemas modernos de IA

**5 equipos – 5 arquitecturas**

Mtro.

17 de septiembre de 2026

# Índice

**1. Objetivo general** **3**

**2. Filosofía del proyecto** **3**

**3. Arquitectura mínima común** **3**

**4. Capas transversales** **5**
4.1. Evaluación . . . 5
4.2. Observabilidad . . . 5
4.3. Seguridad . . . 6

**5. Selección de tecnologías** **6**

**6. Equipo 1 – Sistema universitario inteligente** **8**
6.1. Descripción del problema . . . 8
6.2. Objetivo arquitectónico . . . 8
6.3. Arquitectura esperada . . . 8
6.4. Componentes principales . . . 9
6.5. Tecnologías candidatas . . . 10
6.6. Experimentos obligatorios . . . 10
6.7. Pregunta central . . . 10

**7. Equipo 2 – Sistema de IA para información sensible** **11**
7.1. Descripción del problema . . . 11
7.2. Objetivo arquitectónico . . . 11
7.3. Arquitectura esperada . . . 11
7.4. Componentes principales . . . 12
7.5. Tecnologías candidatas . . . 13
7.6. Pruebas esperadas . . . 13
7.7. Pregunta central . . . 13

**8. Equipo 3 – AI Business Analyst** **14**
8.1. Descripción del problema . . . 14
8.2. Objetivo arquitectónico . . . 14
8.3. Arquitectura esperada . . . 14
8.4. Herramientas que podría utilizar el agente . . . 15
8.5. Componentes principales . . . 15
8.6. Tecnologías candidatas . . . 15
8.7. Requerimiento especial . . . 16
8.8. Pregunta central . . . 16

**9. Equipo 4 – AI Software Engineering System** **17**
9.1. Descripción del problema . . . 17
9.2. Objetivo arquitectónico . . . 17

9.3. Arquitectura propuesta . . . 17
9.4. Roles posibles . . . 18
9.5. MCP . . . 18
9.6. Tecnologías candidatas . . . 19
9.7. Pregunta central . . . 19

**10. Equipo 5 – Plataforma multimodal inteligente** **20**
10.1. Descripción del problema . . . 20
10.2. Objetivo arquitectónico . . . 20
10.3. Arquitectura propuesta . . . 20
10.4. Componentes . . . 21
10.5. Tecnologías candidatas . . . 21
10.6. Experimentos esperados . . . 21
10.7. Pregunta central . . . 22

**11. Requerimientos arquitectónicos comunes** **23**

**12. No todo componente debe utilizarse** **24**

**13. Principio de simplicidad arquitectónica** **24**

**14. Preguntas para la defensa arquitectónica** **25**

**15. Entregables** **27**
15.1. Documento de arquitectura . . . 27
15.2. Diagrama arquitectónico . . . 27
15.3. Implementación funcional . . . 28
15.4. Repositorio . . . 28
15.5. Evaluación . . . 28
15.6. Observabilidad . . . 29
15.7. Análisis de seguridad . . . 29

**16. Presentación final** **29**

**17. Defensa arquitectónica** **30**

**18. Criterios sugeridos de evaluación** **31**

**19. Especialización de cada equipo** **32**

**20. Visión conjunta** **32**

**21. Criterio fundamental** **33**

**22. Resultado esperado** **33**

# 1. Objetivo general

El proyecto final tiene como objetivo que los estudiantes integren los conocimientos adquiridos durante el curso mediante el diseño, justificación, implementación y evaluación de una **arquitectura moderna de Inteligencia Artificial**.

El objetivo principal no será únicamente desarrollar una aplicación que utilice un modelo de Inteligencia Artificial. El estudiante deberá demostrar que comprende cómo los distintos componentes de un sistema de IA interactúan para construir una solución completa, escalable, observable, evaluable y segura.

**Idea clave**

El proyecto no consiste únicamente en construir un chatbot.
El proyecto consiste en diseñar, implementar y defender un **sistema completo de Inteligencia Artificial**.

# 2. Filosofía del proyecto

Los cinco equipos desarrollarán sistemas diferentes.

Sin embargo, todos deberán demostrar dominio de una **base conceptual y arquitectónica común**.

Esto permitirá evaluar las mismas competencias mientras cada equipo enfrenta problemas arquitectónicos distintos.

La arquitectura deberá diseñarse siguiendo el principio:

$$\text{Problema} \rightarrow \text{Requerimientos} \rightarrow \text{Arquitectura} \rightarrow \text{Tecnologías} \rightarrow \text{Implementación}$$

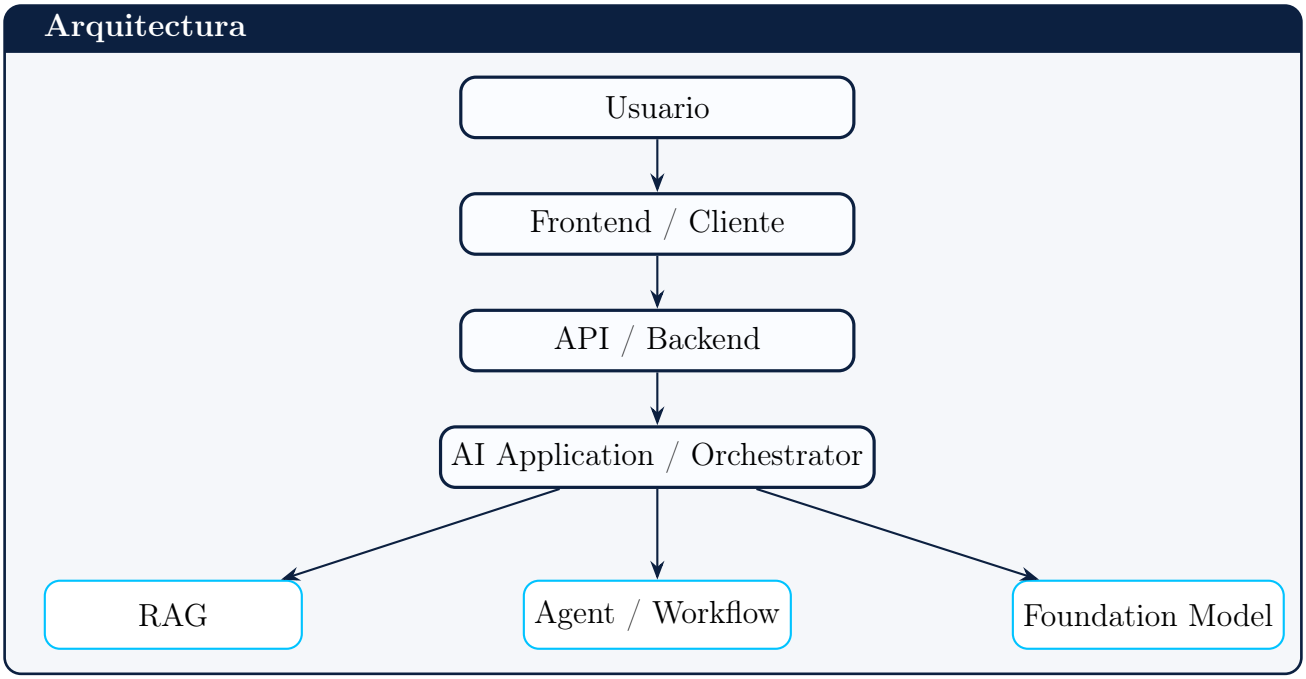
Los estudiantes deberán evitar el proceso inverso:

*"Tengo una herramienta, ahora buscaré dónde utilizarla."*

# 3. Arquitectura mínima común

Todos los equipos deberán estudiar una arquitectura general de sistemas modernos de IA.

No significa que absolutamente todos los componentes tengan que utilizarse en todos los proyectos. Cada equipo deberá determinar cuáles son necesarios y justificar sus decisiones.




La arquitectura completa podrá considerar:

- Interfaz de usuario o cliente.
- Backend.
- API.
- Foundation Model.
- Model Serving.
- Almacenamiento tradicional.
- Embeddings.
- Vector Database.
- Retrieval-Augmented Generation (RAG).
- Structured Outputs.
- Tool / Function Calling.
- Agent o Agentic Workflow.
- State.
- Memory.
- MCP.
- Multi-Agent Systems.
- Evaluación.
- Observabilidad.

- Guardrails.
- Human-in-the-Loop.
- Contenedores.
- Despliegue.
- Escalabilidad.

# 4. Capas transversales

Además de los componentes funcionales, todos los equipos deberán considerar tres dimensiones transversales.

## 4.1 Evaluación

El sistema deberá demostrar cómo puede evaluarse su comportamiento.
Dependiendo del proyecto podrán utilizarse métricas relacionadas con:

- Retrieval.
- Relevance.
- Groundedness.
- Faithfulness.
- Task Completion.
- Tool-use Success.
- Latencia.
- Costo.
- Errores.

## 4.2 Observabilidad

El equipo deberá explicar cómo observaría el sistema en producción.
Deberán considerar:

- Logs.
- Metrics.
- Traces.
- LLM Calls.
- Retrieval Traces.
- Tool Calls.

- Agent Steps.
- Tokens.
- Latencia.
- Costo.
- Errores.

### 4.3 Seguridad

Cada arquitectura deberá realizar un análisis básico de seguridad considerando, cuando corresponda:

- Prompt Injection.
- Indirect Prompt Injection.
- Data Leakage.
- PII.
- Tool Abuse.
- Excessive Agency.
- Control de permisos.
- Autorización.
- Validación de entradas y salidas.
- Human-in-the-Loop.

## 5. Selección de tecnologías

Los equipos **no recibirán todas las tecnologías preseleccionadas**.
Cada equipo deberá investigar alternativas y justificar su elección.
Por ejemplo, si el sistema necesita almacenamiento vectorial, el equipo podrá considerar:

- Qdrant.
- pgvector.
- Pinecone.
- Weaviate.
- Milvus.
- Chroma.

La decisión deberá responder:

1. ¿Qué problema resuelve este componente?

2. ¿Por qué es necesario?
3. ¿Qué alternativas existen?
4. ¿Por qué seleccionaron esta tecnología?
5. ¿Qué ventajas ofrece?
6. ¿Qué limitaciones introduce?
7. ¿Qué ocurriría si se elimina?
8. ¿Cómo cambiaría la arquitectura al crecer el sistema?

**Idea clave**

No gana el equipo que utilice más tecnologías.
Una buena arquitectura utiliza la menor complejidad necesaria para satisfacer correctamente sus requerimientos.

# 6. Equipo 1 – Sistema universitario inteligente

## 6.1 Descripción del problema

Construir un sistema capaz de responder preguntas utilizando información universitaria proveniente de documentos institucionales.

El sistema podrá trabajar con:

- Reglamentos.
- Planes de estudio.
- Programas de materias.
- Calendarios.
- Procedimientos administrativos.
- Documentación académica.
- Preguntas frecuentes.

## 6.2 Objetivo arquitectónico

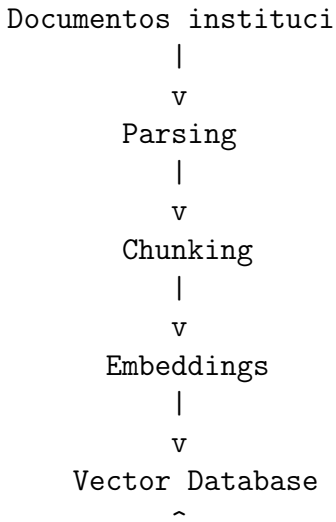
El principal reto será construir una arquitectura **RAG robusta, trazable y evaluable**.

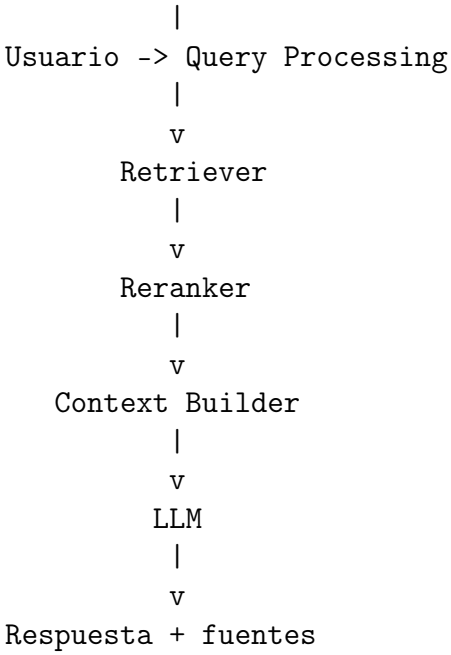
El sistema deberá recuperar información relevante, proporcionarla al modelo como contexto y generar respuestas sustentadas en las fuentes recuperadas.

**Reto arquitectónico**

El reto principal no es conectar un PDF a un LLM.
El equipo deberá demostrar que comprende todo el pipeline de Retrieval-Augmented Generation y que puede evaluar la calidad del proceso de recuperación.

## 6.3 Arquitectura esperada






## 6.4 Componentes principales

- Document Ingestion.
- Parsing.
- Chunking.
- Embeddings.
- Vector Database.
- Semantic Search.
- Keyword Search.
- Hybrid Search.
- Metadata Filtering.
- Retrieval.
- Reranking.
- Context Construction.
- LLM.
- Citations.
- RAG Evaluation.

## 6.5 Tecnologías candidatas

El equipo deberá investigar y seleccionar tecnologías entre alternativas como:

- Python.
- FastAPI.
- Qdrant.
- pgvector.
- Pinecone.
- Hugging Face Embeddings.
- OpenAI Embeddings.
- BGE.
- Cohere Rerank.
- LlamaIndex.
- LangChain.
- Ragas.
- DeepEval.
- Langfuse.
- Docker.

## 6.6 Experimentos obligatorios

El equipo deberá comparar al menos dos estrategias de recuperación.
Por ejemplo:

- Semantic Search.
- Hybrid Search.
- Retrieval + Reranking.
- Diferentes estrategias de Chunking.
- Diferentes modelos de Embeddings.

## 6.7 Pregunta central

**Idea clave**

¿Cómo podemos demostrar que la respuesta del modelo realmente está sustentada en la información recuperada?

# 7. Equipo 2 – Sistema de IA para información sensible

## 7.1 Descripción del problema

Diseñar un asistente que trabaje con información perteneciente a un dominio sensible, donde las respuestas incorrectas o las acciones automáticas puedan representar un riesgo.

El proyecto puede utilizar información simulada relacionada con:

- Documentación médica.
- Procedimientos administrativos sensibles.
- Información financiera simulada.
- Documentación legal ficticia.
- Procesos internos de una organización.

El sistema será estrictamente académico.

## 7.2 Objetivo arquitectónico

El reto será analizar cómo cambia una arquitectura de IA cuando aumenta el impacto potencial de un error.

La arquitectura deberá priorizar:

- Seguridad.
- Privacidad.
- Trazabilidad.
- Grounding.
- Control de acceso.
- Validación.
- Auditoría.
- Human-in-the-Loop.

## 7.3 Arquitectura esperada

```
Usuario
   |
   v
Authentication
   |
   v
Authorization
   |
   v
```

```
Input Guardrails
   |
   v
PII Detection
   |
   v
RAG / Knowledge
   |
   v
LLM
   |
   v
Structured Output
   |
   v
Output Validation
   |
   v
Risk Classification
   |
   +---- Bajo riesgo ----> Response
   |
   +---- Alto riesgo ----> Human Approval
```

## 7.4 Componentes principales

- Authentication.
- Authorization.
- RAG.
- PII Detection.
- Input Guardrails.
- Output Guardrails.
- Structured Outputs.
- Risk Classification.
- Human-in-the-Loop.
- Audit Logs.
- Observability.
- Evaluation.

## 7.5 Tecnologías candidatas

- FastAPI.
- Pydantic.
- Qdrant.
- LangGraph.
- Microsoft Presidio.
- Llama Guard.
- Guardrails AI.
- NeMo Guardrails.
- Langfuse.
- Ragas.
- DeepEval.
- Docker.

## 7.6 Pruebas esperadas

El equipo deberá demostrar cómo responde el sistema ante:

- Una consulta normal.
- Una consulta ambigua.
- Una solicitud fuera del dominio.
- Una entrada potencialmente maliciosa.
- Información sensible.
- Una acción que requiere autorización humana.

## 7.7 Pregunta central

**Idea clave**

¿Qué puede hacer automáticamente la IA y qué debe requerir autorización humana?

# 8. Equipo 3 – AI Business Analyst

## 8.1 Descripción del problema

Construir un sistema capaz de analizar información empresarial proveniente de diferentes fuentes.

El sistema podrá trabajar con:

- Documentos.
- Bases de datos.
- Archivos CSV.
- APIs.
- Reportes.
- Indicadores.
- Información estructurada y no estructurada.

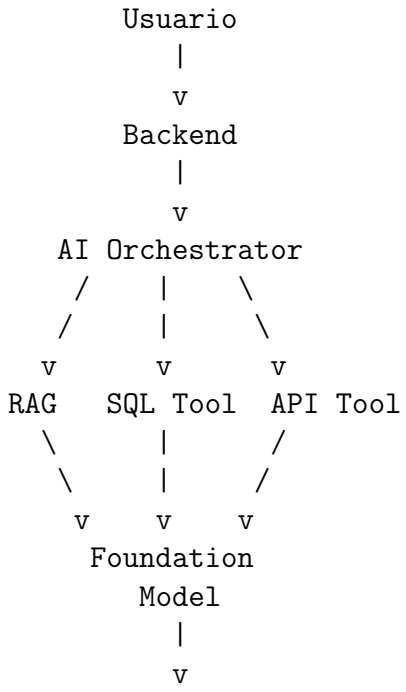
## 8.2 Objetivo arquitectónico

El reto principal será integrar **Foundation Models con herramientas y sistemas externos**.

El modelo no deberá intentar responder todo utilizando únicamente su conocimiento interno.

Deberá seleccionar herramientas dependiendo de la tarea.

## 8.3 Arquitectura esperada



```
Structured Output
        |
        v
Dashboard / Report
```

## 8.4 Herramientas que podría utilizar el agente

El agente podrá disponer de herramientas como:

- SQL Query Tool.
- Calculator.
- Search Tool.
- External API.
- Document Retriever.
- Report Generator.
- Data Analysis Tool.

## 8.5 Componentes principales

- Tool Calling.
- Structured Outputs.
- RAG.
- SQL.
- External APIs.
- Agentic Workflow.
- Model Routing.
- Error Handling.
- Observability.
- Evaluation.

## 8.6 Tecnologías candidatas

- FastAPI.
- Pydantic.
- LangGraph.
- PostgreSQL.
- Qdrant.

- Function Calling.
- MCP.
- Langfuse.
- Docker.

## 8.7 Requerimiento especial

El sistema deberá demostrar al menos tres tipos diferentes de herramientas.
Por ejemplo:

1. Recuperación de información.
2. Ejecución de cálculos.
3. Consulta a una base de datos.

## 8.8 Pregunta central

**Idea clave**

¿Cómo decide el sistema cuándo responder directamente y cuándo utilizar una herramienta externa?

# 9. Equipo 4 – AI Software Engineering System

## 9.1 Descripción del problema

Construir un sistema de IA capaz de colaborar en tareas relacionadas con ingeniería de software.

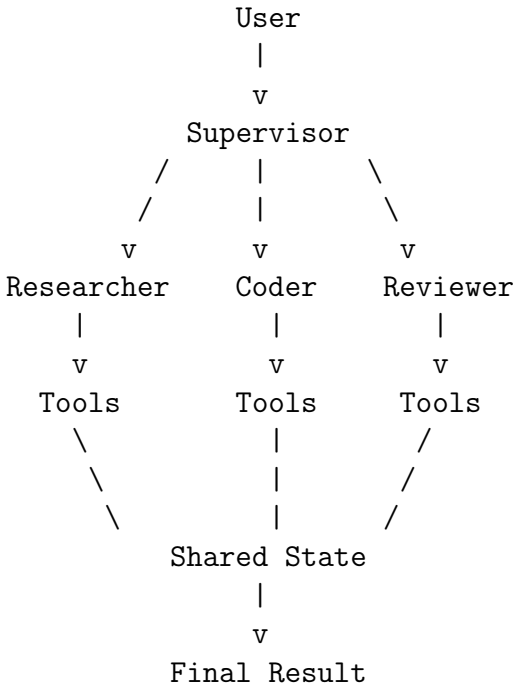
El sistema deberá dividir problemas complejos entre diferentes roles, workflows o agentes especializados.

## 9.2 Objetivo arquitectónico

Este equipo estudiará principalmente:

- Agentic Systems.
- Multi-Agent Architectures.
- MCP.
- Tool Calling.
- Coordinación.
- State.
- Memory.
- Human-in-the-Loop.

## 9.3 Arquitectura propuesta



## 9.4 Roles posibles

**Supervisor**

- Analiza la tarea.
- Divide el problema.
- Selecciona especialistas.
- Asigna responsabilidades.
- Coordina la ejecución.
- Determina cuándo finalizar.

**Research Agent**

- Consulta documentación.
- Utiliza RAG.
- Recupera contexto.
- Consulta herramientas externas.

**Coding Agent**

- Propone implementaciones.
- Genera código.
- Utiliza herramientas.
- Trabaja con archivos.

**Review Agent**

- Revisa resultados.
- Detecta errores.
- Solicita correcciones.
- Evalúa criterios definidos.

## 9.5 MCP

El equipo deberá estudiar cómo un agente puede conectarse a capacidades externas mediante Model Context Protocol.

Podrán analizar servidores MCP relacionados con:

- Archivos.
- Repositorios.

- Bases de datos.
- Documentación.
- APIs.
- Herramientas de desarrollo.

## 9.6 Tecnologías candidatas

- LangGraph.
- MCP.
- Git.
- GitHub.
- FastAPI.
- Qdrant.
- Docker.
- Langfuse.
- CrewAI como comparación.
- AutoGen como comparación.
- PydanticAI como comparación.

## 9.7 Pregunta central

**Idea clave**

¿Cuándo se justifica utilizar varios agentes y cuándo un único workflow sería más simple, barato y confiable?

# 10. Equipo 5 – Plataforma multimodal inteligente

## 10.1 Descripción del problema

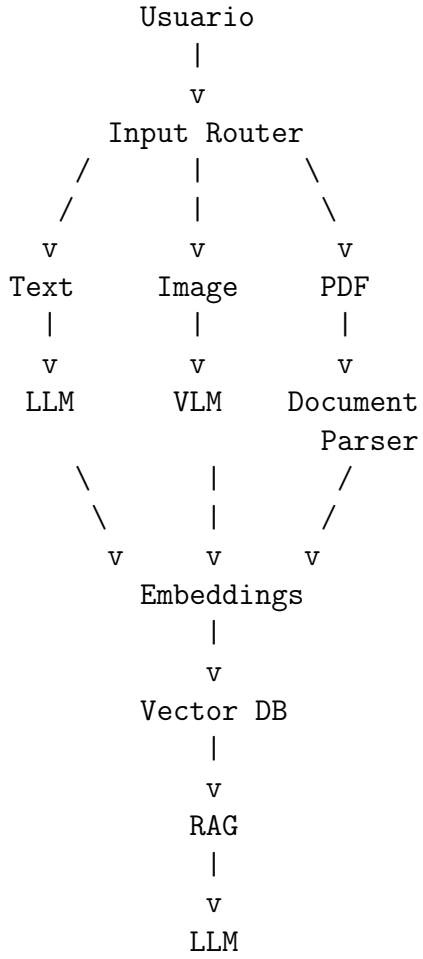
Construir un sistema capaz de procesar distintas modalidades de información.
El usuario podrá proporcionar:

- Texto.
- Imágenes.
- PDFs.
- Documentos.
- Opcionalmente audio.

## 10.2 Objetivo arquitectónico

El reto consiste en diseñar diferentes pipelines dependiendo del tipo de información recibida y posteriormente integrar la información en una experiencia común.

## 10.3 Arquitectura propuesta



```
        |
        v
    Response
```

## 10.4 Componentes

- Input Router.
- Document Parsing.
- Vision-Language Model.
- Embeddings.
- Multimodal Embeddings.
- Vector Database.
- Multimodal RAG.
- Model Routing.
- Structured Outputs.
- Evaluation.
- Observability.

## 10.5 Tecnologías candidatas

- FastAPI.
- Modelos multimodales vía API.
- Modelos open-weight.
- Hugging Face.
- Qdrant.
- LlamaIndex.
- LangGraph.
- Langfuse.
- Docker.

## 10.6 Experimentos esperados

El equipo deberá mostrar cómo cambia el pipeline dependiendo de la entrada.
Por ejemplo:

- Texto.
- Documento.

- Imagen.
- Documento con imágenes.
- Consulta que requiere combinar modalidades.

## 10.7 Pregunta central

**Idea clave**

¿Cómo debe enrutar el sistema cada modalidad y cómo puede integrar posteriormente la información obtenida?

# 11. Requerimientos arquitectónicos comunes

Todos los equipos deberán analizar los siguientes componentes.
Para cada elemento deberán indicar:

- Si su arquitectura lo utiliza.
- Qué función cumple.
- Qué tecnología lo implementa.
- Por qué fue seleccionado.
- Si no se utiliza, por qué no es necesario.

Los componentes a analizar son:

1. Cliente o Frontend.
2. Backend.
3. API.
4. Foundation Model.
5. Estrategia de inferencia.
6. Almacenamiento.
7. Embeddings.
8. Vector Database.
9. RAG.
10. Structured Outputs.
11. Tool Calling.
12. Agent o Workflow.
13. State.
14. Memory.
15. MCP.
16. Multi-Agent.
17. Evaluation.
18. Observability.
19. Security / Guardrails.
20. Human-in-the-Loop.
21. Containerization.

22. Deployment.
23. Scalability.

## 12. No todo componente debe utilizarse

No todos los sistemas necesitan todos los componentes.
Por ejemplo:

- No todo sistema necesita Multi-Agent.
- No todo sistema necesita MCP.
- No todo sistema necesita Fine-Tuning.
- No todo sistema necesita GraphRAG.
- No todo sistema necesita Kubernetes.
- No todo sistema necesita una Vector Database.
- No todo sistema necesita memoria persistente.
- No todo sistema necesita un agente.

Cuando un componente no sea utilizado, el equipo deberá ser capaz de responder:

### ¿Por qué nuestra arquitectura no lo necesita?

## 13. Principio de simplicidad arquitectónica

**Idea clave**

Agregar más tecnologías no significa tener una mejor arquitectura.

Una arquitectura deberá utilizar solamente los componentes necesarios para satisfacer sus requerimientos.

Por tanto, no se premiará el uso indiscriminado de:

- Múltiples frameworks.
- Múltiples agentes.
- Múltiples bases de datos.
- Múltiples modelos.
- Protocolos innecesarios.
- Infraestructura innecesariamente compleja.

Toda complejidad adicional deberá estar justificada.

# 14. Preguntas para la defensa arquitectónica

Durante la presentación final, cualquier integrante del equipo deberá poder responder preguntas como:

1. ¿Cuál es el problema que resuelve su sistema?
2. ¿Cuáles son sus requerimientos funcionales?
3. ¿Cuáles son sus requerimientos no funcionales?
4. ¿Dónde se encuentra la Inteligencia Artificial?
5. ¿Qué modelo utilizan?
6. ¿El modelo es local o remoto?
7. ¿Por qué?
8. ¿Cómo realizan inferencia?
9. ¿Cómo manejan conocimiento externo?
10. ¿Necesitan RAG?
11. ¿Cómo funciona su pipeline RAG?
12. ¿Qué estrategia de Retrieval utilizan?
13. ¿Qué modelo de Embeddings utilizan?
14. ¿Qué Vector Database eligieron?
15. ¿Por qué?
16. ¿Utilizan Reranking?
17. ¿Cómo manejan el contexto?
18. ¿Utilizan Structured Outputs?
19. ¿Utilizan herramientas?
20. ¿Cómo funciona Tool Calling?
21. ¿Utilizan agentes?
22. ¿Por qué necesitan un agente?
23. ¿Cómo manejan State?
24. ¿Cómo manejan Memory?
25. ¿Utilizan MCP?
26. ¿Necesitan múltiples agentes?
27. ¿Cómo evalúan las respuestas?

28. ¿Cómo detectan errores?
29. ¿Cómo observan el sistema?
30. ¿Cómo miden latencia?
31. ¿Cómo miden costo?
32. ¿Qué amenazas de seguridad existen?
33. ¿Qué Guardrails implementaron?
34. ¿Qué acciones requieren Human-in-the-Loop?
35. ¿Cómo desplegarían el sistema?
36. ¿Cómo escalaría con 100 usuarios?
37. ¿Cómo escalaría con 10,000 usuarios?
38. ¿Qué componente se convertiría primero en cuello de botella?
39. ¿Qué cambiarían para llevar el sistema a producción real?

# 15. Entregables

Cada equipo deberá entregar los siguientes elementos.

## 15.1 Documento de arquitectura

El documento deberá contener:

- Descripción del problema.
- Alcance.
- Requerimientos funcionales.
- Requerimientos no funcionales.
- Arquitectura general.
- Descripción de componentes.
- Flujo de datos.
- Flujo de inferencia.
- Decisiones tecnológicas.
- Alternativas consideradas.
- Trade-offs.
- Riesgos.
- Limitaciones.
- Estrategia de escalabilidad.

## 15.2 Diagrama arquitectónico

El diagrama deberá mostrar claramente:

- Usuarios.
- Clientes.
- Servicios.
- Modelos.
- Bases de datos.
- Vector Databases.
- APIs.
- Agentes.
- Herramientas.

- Servicios externos.
- Flujo de información.
- Límites del sistema.

## 15.3 Implementación funcional

El sistema deberá incluir una implementación funcional de los componentes principales.
No será obligatorio implementar infraestructura de escala empresarial completa, pero sí deberán explicar cómo evolucionaría la arquitectura para operar en producción.

## 15.4 Repositorio

El repositorio deberá contener:

- Código fuente.
- README.
- Diagrama de arquitectura.
- Instrucciones de instalación.
- Dependencias.
- Variables de configuración documentadas.
- Dockerfile.
- Documentación técnica.

Las credenciales, API Keys y secretos no deberán almacenarse directamente en el repositorio.

## 15.5 Evaluación

Cada equipo deberá definir métricas adecuadas para su sistema.
Podrán incluir:

- Calidad.
- Retrieval.
- Relevance.
- Groundedness.
- Faithfulness.
- Task Completion.
- Tool-use Success.
- Latencia.
- Costo.

- Tasa de errores.
- Disponibilidad.

### 15.6 Observabilidad

El sistema deberá permitir observar, cuando corresponda:

- Llamadas al modelo.
- Prompts.
- Retrieval.
- Tools.
- Agentes.
- Tokens.
- Latencia.
- Errores.
- Costos.

### 15.7 Análisis de seguridad

Cada equipo deberá realizar un análisis básico de amenazas que considere:

- Prompt Injection.
- Indirect Prompt Injection.
- Data Leakage.
- PII.
- Tool Abuse.
- Permisos.
- Acciones automatizadas.
- Accesos no autorizados.
- Protección de secretos.
- Human-in-the-Loop.

## 16. Presentación final

Cada equipo deberá presentar:

1. Problema.

2. Alcance.
3. Requerimientos.
4. Arquitectura.
5. Flujo de información.
6. Decisiones tecnológicas.
7. Alternativas consideradas.
8. Demostración funcional.
9. Evaluación.
10. Observabilidad.
11. Seguridad.
12. Escalabilidad.
13. Limitaciones.
14. Mejoras futuras.

## 17. Defensa arquitectónica

Una parte fundamental de la evaluación será la **defensa de la arquitectura**.
El profesor podrá seleccionar cualquier componente del diagrama y preguntar:

# ¿Por qué está aquí?

El estudiante deberá explicar:

1. Qué problema resuelve.
2. Qué información recibe.
3. Qué información produce.
4. Con qué componentes se comunica.
5. Qué tecnología utilizaron.
6. Por qué seleccionaron esa tecnología.
7. Qué alternativas existen.
8. Qué ventajas ofrece.
9. Qué limitaciones introduce.
10. Qué ocurriría si el componente se elimina.

# 18. Criterios sugeridos de evaluación

| **Criterio** | **Ponderación** |
|---|---|
| Comprensión del problema y requerimientos | 10 % |
| Diseño de arquitectura | 20 % |
| Justificación de decisiones tecnológicas | 15 % |
| Implementación funcional | 20 % |
| Evaluación del sistema | 10 % |
| Observabilidad | 5 % |
| Seguridad y análisis de riesgos | 5 % |
| Documentación y repositorio | 5 % |
| Presentación y defensa arquitectónica | 10 % |
| **Total** | **100 %** |

## 19. Especialización de cada equipo

| Equipo | Sistema | Especialización arquitectónica |
|---|---|---|
| 1 | Sistema universitario inteligente | RAG, Retrieval, Embeddings, Vector Databases, Reranking y evaluación. |
| 2 | Sistema para información sensible | Guardrails, privacidad, seguridad, trazabilidad y Human-in-the-Loop. |
| 3 | AI Business Analyst | Tool Calling, APIs, datos estructurados, agentes y Structured Outputs. |
| 4 | AI Software Engineering System | Agentic Workflows, Multi-Agent, MCP, State y coordinación. |
| 5 | Plataforma multimodal | VLM, documentos, Multimodal RAG, routing y múltiples modalidades. |

## 20. Visión conjunta

Los cinco proyectos representan diferentes especializaciones dentro de una arquitectura moderna de sistemas de IA.

**Equipo 1**
RAG / Retrieval

↓

**Equipo 2**
Safety / HITL

↓

**Equipo 3**
Tools / Integration

↓

**Equipo 4**
Agents / MCP / Multi-Agent

↓

**Equipo 5**

Multimodal AI

De esta forma, aunque cada estudiante implemente profundamente una arquitectura particular, las exposiciones finales permiten que el grupo estudie cinco variantes diferentes de sistemas modernos de Inteligencia Artificial.

## 21. Criterio fundamental

**Idea clave**

El proyecto no será evaluado por la cantidad de tecnologías utilizadas.
Será evaluado por la calidad de las **decisiones arquitectónicas**.

Un sistema pequeño, bien diseñado, evaluado y correctamente justificado puede representar una mejor arquitectura que un sistema con muchos componentes innecesarios.

## 22. Resultado esperado

Al finalizar el proyecto, el estudiante deberá poder observar una arquitectura moderna de Inteligencia Artificial e identificar:

- Dónde están los datos.
- Dónde está el modelo.
- Cómo ocurre la inferencia.
- Cómo se incorpora conocimiento externo.
- Cómo funciona RAG.
- Cómo funcionan los Embeddings.
- Para qué sirve una Vector Database.
- Cómo funcionan las herramientas.
- Cómo funciona Tool Calling.
- Cómo funcionan los Agentic Workflows.
- Cómo funcionan los agentes.
- Cómo se maneja State.
- Cómo se maneja Memory.
- Qué función puede cumplir MCP.

- Cuándo tiene sentido Multi-Agent.
- Cómo se evalúa el sistema.
- Cómo se observa.
- Cómo se protege.
- Cómo se despliega.
- Cómo puede escalar.

**Idea clave**

El objetivo final de la materia no es que el estudiante sepa utilizar una herramienta específica.
El objetivo es que pueda **diseñar, construir, evaluar y defender la arquitectura de un sistema moderno de Inteligencia Artificial**.

## Arquitectura de Sistemas de Inteligencia Artificial

Del modelo aislado al sistema completo.